NIRMA UNIVERSITY

Institute Of technology,Nirma University

Project Report of

Design and Implementation of a 16-bit ALU with Status Flags using Verilog HDL on FPGA

Submitted to:

Prof.Rutul Patel

Prof. Aakash Mecwan

Submitted By:

Manash Gusani(25BEC505)

Henil Patel(25BEC518)

Subject:FPGA

Semester:4th

Batch:DtoD

Date:13/03/2026

Design and Implementation of a 16-bit ALU with Status Flags using Verilog HDL on FPGA

1. Aim

To design and implement a 16-bit Arithmetic Logic Unit (ALU) using Verilog HDL, simulate its functionality using ModelSim, and implement it on an FPGA board using Quartus.

2. Objectives

- To understand the working of an Arithmetic Logic Unit.
- To design combinational logic using Verilog HDL.
- To verify the ALU operations using RTL simulation.
- To implement the ALU on FPGA hardware.
- To observe arithmetic and logical operations using FPGA switches and LEDs.

3. Theory

An **Arithmetic Logic Unit (ALU)** is the fundamental computational component of a processor. It performs arithmetic operations such as addition and subtraction and logical operations such as AND, OR, and XOR.

The ALU takes two input operands and performs the operation selected by a control signal. In this design, a **4-bit selection input (ALU_Sel)** determines which operation is performed.

The ALU also generates status flags indicating the condition of the result.

4.Status Flags Generated by the ALU

The ALU produces **six status flags** that describe the condition of the result after performing arithmetic or logical operations. These flags are important in processors because they help control conditional operations and detect arithmetic conditions such as overflow or zero results.

Flag	Name	Description
CF	Carry Flag	Indicates carry out of the most significant bit during addition or borrow during subtraction
OF	Overflow Flag	Indicates overflow in signed arithmetic operations
PF	Parity Flag	Set when the least significant byte of the result contains an even number of 1s
SF	Sign Flag	Indicates the sign of the result (MSB of the result)
ZF	Zero Flag	Set when the result of the operation is zero
AF	Auxiliary Carry Flag	Indicates carry from bit 3 to bit 4 (used in BCD arithmetic)

5. Code

```
module ALU(
   input [1:0] A_in,        // user gives 2 MSB bits
   input [1:0] B_in,        // user gives 2 MSB bits
   input [3:0] ALU_Sel,
         input notA,
         input notB,
   output reg [15:0] Result,
   output reg CF, ZF, SF, OF, PF, AF );
```

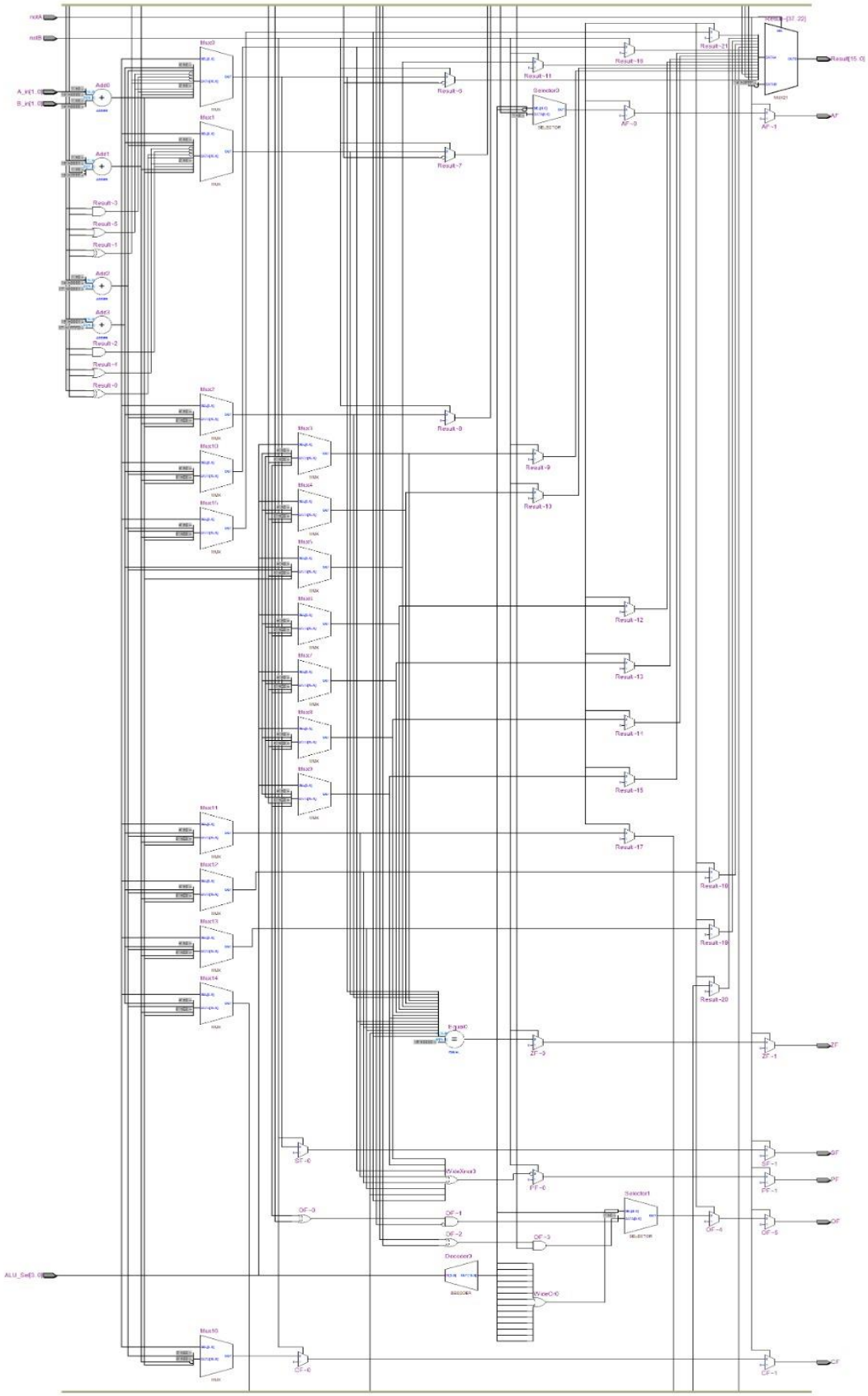
```
// Construct full 16-bit operands
wire [15:0] A;
wire [15:0] B;
assign A = {A_in, 14'b00000000000000};
assign B = {B_in, 14'b00000000000000};
reg [16:0] temp;
always @(*)
begin
Result = 16'b0;
CF = 0;
OF = 0;
AF = 0;
ZF = 0;
SF = 0;
PF = 0;
if (notA)
begin
   Result = ~A;
end
else if (notB)
begin
   Result = ~B;
end
else
begin
case(ALU_Sel)
4'b0000: begin   // ADD
   temp = A + B;
Result = temp[15:0];
   CF = temp[16];
   AF = (A[3] & B[3]) | ((A[3] ^ B[3]) & ~Result[3]);
   OF = (A[15] == B[15]) && (Result[15] != A[15]);
end
4'b0001: begin   // SUB
   temp = A - B;
   Result = temp[15:0];
   CF = temp[16];
   AF = (A[3] ^ B[3] ^ Result[3]);
   OF = (A[15] != B[15]) && (Result[15] != A[15]);
end
4'b0010: Result = A & B;      // AND
4'b0011: Result = A | B;      // OR
4'b0110: Result = A ^ B;      // XOR
4'b0111: Result = ~(A^B);     // XNOR
4'b1000: Result = ~(A & B);   // NAND
4'b1001: Result = ~(A | B);   // NOR
4'b1010: begin                // Increment
   temp = A + 1;
   Result = temp[15:0];
   CF = temp[16];
end
4'b1011: begin                // Decrement
   temp = A - 1;
   Result = temp[15:0];
end
4'b1100: Result = A << 1;     // Shift Left
```

```
4'b1101: Result = A >> 1;      // Shift Right
default: Result = 16'b0;
endcase
// Zero Flag
ZF = (Result == 16'b0);
// Sign Flag
SF = Result[15];
// Parity Flag
PF = ~^Result[7:0];
end
end
endmodule
```

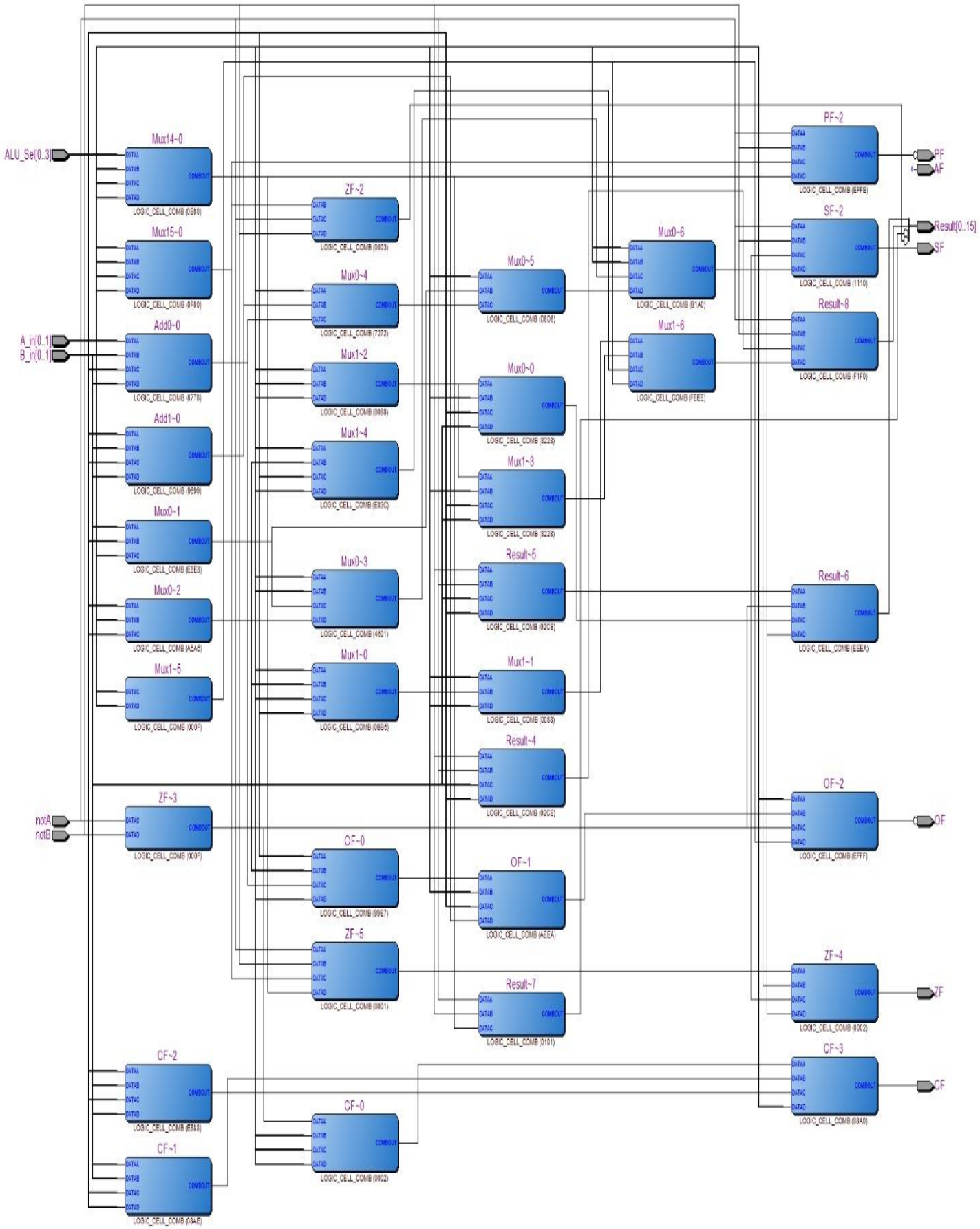
6. ALU Operation Selection Table

Sr. No.	Operation	ALU_Sel (4-bit)
1	ADD	0000
2	SUB	0001
3	AND	0010
4	OR	0011
5	NOT A	*(SW[8] = 1)*
6	NOT B	*(SW[9] = 1)*
7	XOR	0110
8	XNOR	0111
9	NAND	1000
10	NOR	1001
11	INC	1010
12	DEC	1011
13	SHIFT LEFT	1100
14	SHIFT RIGHT	1101

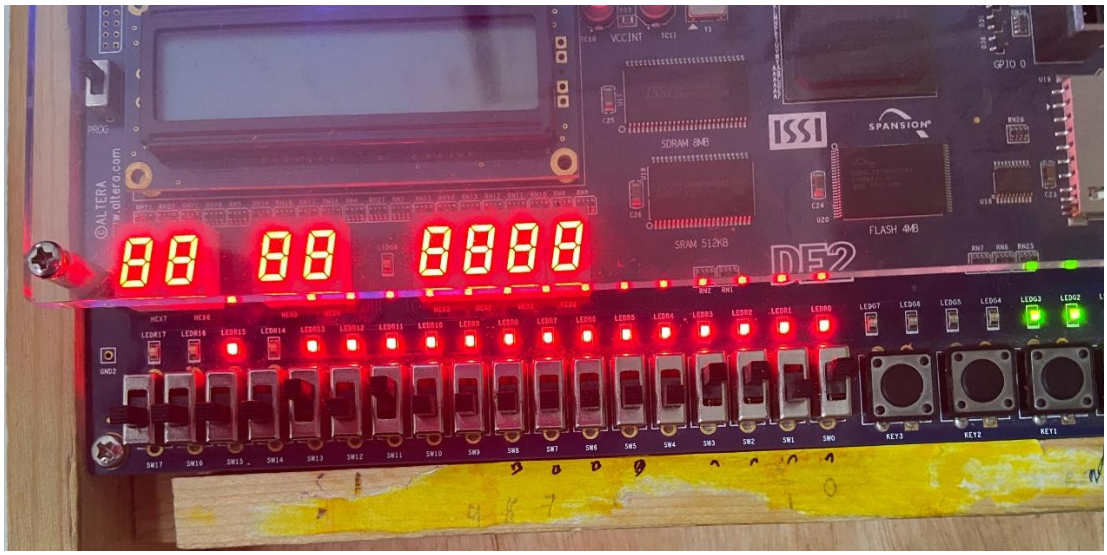
7.RTL VIEW

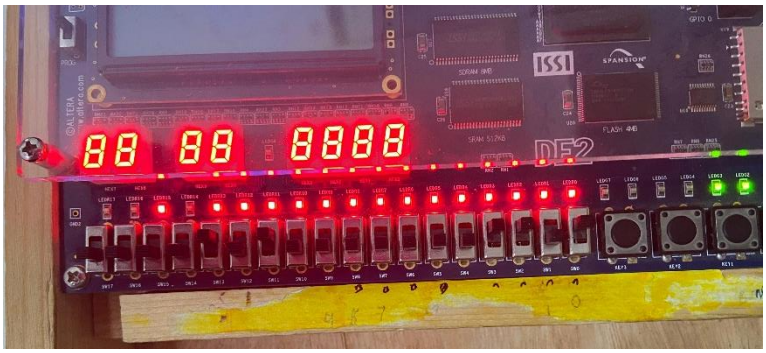


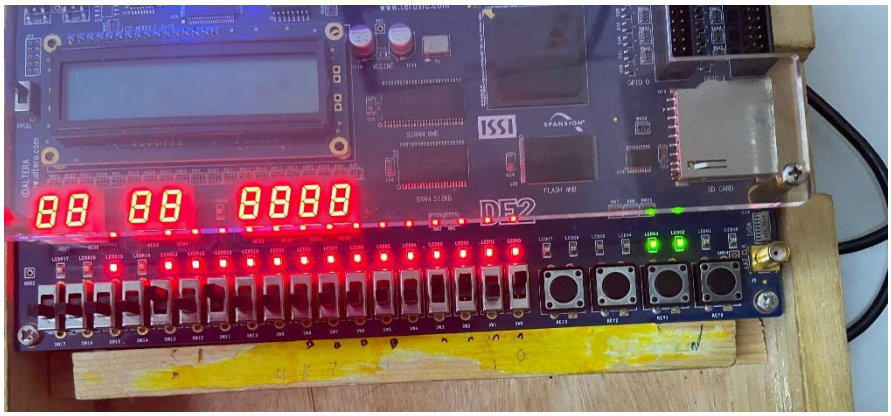
8.TTL VIEW


ALU_Sel[0..3]
A_in[0..1]
B_in[0..1]
notA
notB
Mux14~0
Mux15~0
Add0~0
Add1~0
Mux0~1
Mux0~2
Mux1~5
ZF~3
CF~2
CF~1
ZF~2
Mux0~4
Mux1~2
Mux1~4
Mux0~3
Mux1~0
OF~0
ZF~5
CF~0
Mux0~5
Mux0~0
Mux1~3
Result~5
Mux1~1
Result~4
OF~1
Result~7
Mux0~6
Mux1~6
PF~2
SF~2
Result~8
Result~6
OF~2
ZF~4
CF~3
PF
AF
Result[0..15]
SF
OF
ZF
CF

9. OUTPUT







https://drive.google.com/file/d/1C3430nCYWdqU_0969zL4QbFIdnLu5Fef/view?usp=drive_link

Conclusion

The 16-bit Arithmetic Logic Unit (ALU) was successfully designed using Verilog HDL and its functionality was verified through RTL simulation. Various arithmetic, logical, and shift operations were implemented using the 4-bit control signal ALU_Sel. The simulation results confirmed that all operations produced the correct outputs. The ALU also generated the status flags CF, OF, PF, SF, ZF, and AF, which indicate the condition of the result. The design was then synthesized and implemented on the FPGA board using Quartus. FPGA switches were used to provide input values and select operations, while LEDs displayed the output results. Both simulation and hardware implementation confirmed the correct operation of the ALU. Therefore, the designed ALU can be used as an important building block in digital systems and processor architectures.